AO-02 Capacity Observatory

Complete project guide | Current source snapshot | 29 October 2026

1. Purpose and scope

AO-02 identifies data-center capacity risks, prioritizes resource incidents and produces multi-agent advice for human review. It monitors power, thermal/cooling, storage, network, VM compute and software licenses. It is an advisory application: approval records a decision and does not execute live infrastructure changes.

2. Architecture

React, TypeScript, Vite and Tailwind CSS implement the frontend. FastAPI provides the Python backend; Pydantic validates models, NumPy supports forecasts and LangGraph coordinates agents. REST APIs carry requests and snapshots. WebSocket events update queues and phase transitions. The shared frontend store owns communication, selected incidents, loading state and job start timestamps.

3. End-to-end flow

Forecast CSV upload -> validation and unit conversion -> replacement of supplied telemetry series -> immediate resource scan -> token and context creation -> priority queue -> operator-triggered agent workflow -> root cause, options and risk -> human decision -> decision history. Uploading queues breached resources automatically; it does not automatically run every queued token through a model.

4. Installation

Use Python 3.11 for the pinned dependency set and Node.js 20+. In the extracted project folder, create a virtual environment with python -m venv .venv and activate .\.venv\Scripts\Activate.ps1. Install pip install -r backend/requirements.txt. Create backend/.env and set LLM_PROVIDER=mock for a demonstration without API keys. Start python -m uvicorn app.main:app --app-dir backend --host 127.0.0.1 --port 8017.

5. Frontend launch

In a second PowerShell terminal, enter frontend and run npm ci. Set $env:VITE_API_BASE="http://127.0.0.1:8017" and run npm run dev -- --host 127.0.0.1 --port 5175. Open http://127.0.0.1:5175. Backend API documentation is at http://127.0.0.1:8017/docs. Vite embeds the API URL during production builds; rebuild with npm run build when changing that URL. npm run preview serves the built frontend.

6. Facility screen

The isometric map shows rack and cooling readings, selection inspectors and incident overlays. Zoom and reset affect presentation. Server lights, pipe flow and airflow are visual animations; they are not measurements of individual hardware activity. Facility summaries combine latest readings and open incident state.

7. Forecast screen

Select a node, tenant and resource to inspect its telemetry window, utilization, threshold and projection. Custom telemetry appears here only. The example CSV provides a starting format. Upload & scan CSV imports the data, checks the supplied series and reports the number of new tokens queued. Existing matching queued incidents are updated rather than duplicated.

8. CSV structure

Use UTF-8 CSV, optionally with a BOM. Maximum upload size is 10 MiB. Comma, semicolon, tab and pipe delimiters are attempted. Headings are normalized for spacing, punctuation and capitalization. Common aliases include Timestamp/Time/Recorded At, Rack/Server/Node/Host, Tenant/Customer/Client, Resource/Metric and Value/Reading. Heading recognition is rule based; unknown or ambiguous fields may need renaming.

9. CSV layouts and identifiers

Long layout uses timestamp,rack_id,tenant_id,resource_type,value,unit with one resource per row. Wide layout uses separate resource columns such as Power (kW) and Storage (TB). Do not mix layouts. Supported IDs are R-01 through R-12, CRAC-A-North, CRAC-B-South and FACILITY; tenants are T1-T3 and FACILITY. Common rack/tenant spelling variants are normalized. Use ISO timestamps such as 2026-10-29T10:00:00Z; missing timezones are interpreted as UTC.

10. Units and storage capacity

Canonical units are kW, Celsius, %, Gbps, VMs and Seats. Accepted conversions include watts/megawatts to kW, Mbps/Kbps/Tbps to Gbps and Fahrenheit/Kelvin to Celsius. VM/VMs/virtual machines and Seat/License variants are accepted. Storage in TB, GB or binary units requires total capacity. Supply a Storage Capacity column or the optional upload capacity field in TB. CSV capacity takes precedence. 8 TB used out of 10 TB becomes 80%; capacity is not guessed.

11. CSV examples

Long format header: timestamp,rack_id,tenant_id,resource_type,value,unit,capacity. Example row: 2026-10-29T10:00:00Z,R-12,T3,STORAGE,9.1,TB,10. Wide format header: Time,Server,Customer,Storage (TB),Storage Capacity (TB). Example row: 2026-10-29T10:00:00Z,R-12,T3,9.1,10. Use multiple timestamps for the same series to support forecasting.

12. Validation and import behavior

Duplicate headings or series timestamps, unknown identifiers, malformed numbers and incompatible units produce errors. Up to ten row errors are reported together. Parsing validation errors prevent those readings from being imported. An upload replaces readings only for supplied node/tenant/resource series and retains other generated or imported series. A short series can flag a current breach, while trend forecasting needs at least ten readings. Imported state is held in memory and resets on backend restart.

13. Resource limits

Configured hard limits and safety margins are: Power 80 kW / 90%; Cooling 27 Celsius / 88%; Storage 100% / 85%; Network 10 Gbps / 80%; VM compute 500 VMs / 80%; Software licenses 1000 Seats / 85%. These are application configuration values, not measurements of an actual facility. Evaluation compares the latest reading with hard limit multiplied by safety-margin percentage.

14. Forecasting

The backend uses linear regression over elapsed time, residual error and R-squared confidence, with transient-spike and regime-change heuristics. Forecasts include lead time, optional exhaustion date, confidence and interval. These are heuristic projections rather than guarantees. The frontend can calculate a simple fallback trend estimate. Window parameters named lookback_hours currently select the latest N points, so irregular CSV sampling does not necessarily represent N actual hours.

15. Priority scoring

The min-heap uses scores from 1 to 100; lower runs first. Capacity-normalized utilization makes different measurement units comparable. P1: capacity reached, or power/cooling at least 95%. P2: other utilization at least 95%, or reliable exhaustion within 24 hours. P3: utilization at least 85%, or reliable exhaustion within seven days. P4: remaining flagged incidents. Reliable timing requires confidence at least 0.6 and a positive runway below the stable sentinel.

16. Priority formula and ordering

Score = 25 x band index + 1 + impact + headroom penalty + runway penalty. Bands have indices 0-3. Resource ranks are Power, Cooling, Storage, Network, VM Compute, Software License. Impact = 3 x (rank - 1). Headroom penalty = floor((100-utilization)/5), bounded to 0-4. Reliable runway penalty = floor(days), bounded to 0-4; otherwise 4. Ties use creation time then token ID. Rescanning a queued breach updates its priority, telemetry and forecast while preserving token identity. This scheduling policy should be calibrated to a real facility.

17. Tokens and shared context

IncidentToken carries ID, priority, reason, node, tenant, resource, phase, hop count and creation time. The shared heap envelope contains telemetry, forecast, root cause, remediation options, risk evaluation, operator decision and audit entries. Queue deduplication uses node, tenant and resource. Pending or historical incidents have separate lifecycle roles from queued work.

18. Agent workflow

The six phases are Resource Monitoring, Incident Reporting, Root Cause Analysis, Remediation Checking, Risk Analysis and Human Verification. Monitoring can suppress transient spikes; reporting dispatches the incident; RCA attributes causes; remediation proposes read-only mitigation options; risk evaluates impact; human review records the final choice. The graph interrupts before the human-verification node. The decision API handles operator actions.

19. Pipeline operation

Running without a token ID selects the queue head. Selecting a specific token permits an operator-directed override of priority order. Failed runs return tokens to the queue when they came from it. The hop guard limits transitions to eight. Pipeline execution is initiated through the UI/API; there is no continuously running worker that automatically consumes every token.

20. Operator and decision history

Review evidence, root cause, remediation options and risk, then approve, reject, defer or resolve as provided by the interface. The timer uses a shared job-start timestamp and survives switching views. Stop scan cancels the frontend request; it is not a guaranteed cancellation protocol for an already-running server model call. Decision history displays readable agent labels while retaining original actor and summary values in stored audit records.

21. AI configuration

core/llm_provider.py implements mock, OpenAI and Gemini providers plus provider selection/fallback behavior. LLM_PROVIDER and provider keys configure the service; agent-specific configuration is also supported. Store credentials in backend/.env. Never include real keys in a shared ZIP. Mock mode supports a repeatable demonstration without an external model provider; real model output and latency depend on service availability and configuration.

22. Security and persistence boundaries

The tenant guard checks heap-view access and translates access errors to HTTP responses. Facility access currently relies on request headers; production identity verification is not implemented. Telemetry SHA-256 values link audit entries to series evidence, but an in-memory history is not a durable tamper-proof audit system. Before production use, add authentication, trusted authorization, durable storage, background-job ownership and operational monitoring.

23. Demonstration checklist

Start both services. Open Forecast, download the example CSV and upload it. Confirm automatic scan results. Open Queue, explain the numeric score and its reason, select a token and run the agent workflow. Review RCA/options/risk in Operator and record a decision. Inspect Decision history. Repeated uploads can report zero new tokens because the existing queued token was refreshed. Restarting creates a clean session but clears all imported and incident state.

24. Troubleshooting

Connection failure: check ports, API base and running services. Empty queue: check current reading, units and configured safety margin. Storage error: supply positive total capacity in a matching unit. Unknown heading/unit: use a recognized alias or convert the source. Low confidence: provide enough regular historical samples. Provider errors: inspect configuration and network access or use mock mode. Refresh the frontend after changes; rebuild production assets when configuration changes.

25. Source map and maintenance

Backend: app/main.py is the entry point; core/ao02_state_and_graph.py defines domain schemas, queue and heap; services/csv_telemetry.py parses CSV; services/ingestion.py evaluates readings; workflow.py wires agents; agents/ contains agent implementations. Frontend: App.tsx selects views; store/ao02Store.tsx owns state; services/api.ts normalizes transport fields; components/ contains the four views; lib/resourceSpec.ts defines presentation limits. Keep frontend and backend resource specifications aligned.

26. Existing verification commands

From backend: python -m unittest discover -s tests -v. From frontend: npm run build and node scripts/check-rack-map.mjs. Existing checks cover selected upload, priority and geometry behaviors, not every production scenario. No new application validation is claimed solely by producing this document.

27. API route reference

POST /agents/run: Run the 6-phase agent pipeline on a specific token or the highest-priority token.

GET /agents/pending: Tokens that reached the human-verification gate and await a decision.

POST /agents/decision: Record a human-in-the-loop decision in the immutable audit trail.

GET /agents/history: Completed incidents (resolved, rejected, or halted), newest first.

GET /facility/overview: facility_overview

GET /heap/{token_id}: Return a tenant-isolated view of a heap envelope (403 on isolation breach).

GET /heap/{token_id}/envelope: Return the full heap envelope (facility-operator / internal use).

POST /ingest/scan: Run the monitoring scan: evaluate all resources, mint tokens, enqueue, heap.

POST /ingest/upload: Import CSV readings, replacing only the supplied telemetry series.

GET /queue/snapshot: Return all queued IncidentTokens ordered by priority (for the visualizer).

POST /queue/enqueue: Manually enqueue an IncidentToken (rejects duplicates).

WEBSOCKET /stream/agent-events: WebSocket stream for live agent graph state transitions.

GET /stream/queue/snapshot: HTTP fallback for the current priority queue snapshot.

POST /telemetry/generate: Generate (or reload) the 12-month synthetic telemetry dataset.

GET /telemetry/points: List telemetry points with optional filtering.

GET /telemetry/window: Return the latest telemetry window for a rack/tenant/resource series.

Also available: GET /, GET /health, /docs and /openapi.json. CSV upload uses a raw request body, not multipart form data. POST /agents/run accepts optional token_id. POST /agents/decision accepts token_id, decision, operator and notes. Refer to /docs for detailed schemas and current validation constraints.

28. Source basis

Prepared from the current application source in this workspace on 29 October 2026. Route names were extracted from router source. This guide documents implemented behavior and distinguishes current limitations from future deployment requirements. No environment secrets are included.